University ERP System

Feature Overview & MongoDB Schema Reference

Feature-level walkthrough and field-level schema definitions for the assigned feature modules.

Sakshi Kognole | September 2026

Part 1 — Feature Overview

1. Book Management

Manages the university library book catalogue. Full CRUD with search, pagination, and mobile-responsive display.

- **List view** — paginated table with columns: Book ID, #, Title, Author, Copies, Location, Department. Configurable items-per-page.
- **Search** — searches across Book Title, Author Name, and Book ID using a debounced search bar.
- **Book ID** — entered by the user on the Add form (e.g. BK-001, CS-101). Validated as unique before saving. Read-only on the Edit form. Existing books without an ID are automatically assigned sequential BK-NNN IDs on startup.
- **Add / Edit form** — collects Book ID, Title, Author Name, Total Copies, Book Location, and Department (dropdown: Artificial Intelligence / Computer Science / Civil / Electronics).
- **Duplicate validation** — before saving, the full database is searched for an existing book with the same Title + Author. If found, a warning banner with an OK button is shown inside the modal. Book ID uniqueness is also validated.
- **Enter key submit** — pressing Enter from any field (except textarea) submits the form.
- **View modal** — displays all book details including the correct human-readable Book ID.
- **Delete** — confirmation dialog before deletion.
- **Mobile** — on screens ≤ 768px, books display as accordion cards instead of a table. Tapping a card expands it to show all details and action buttons. Only one card is open at a time.
- **Skeleton loader** — while data loads, a shimmer skeleton matching the table structure is shown instead of a blank screen.

2. Club Management

Manages university clubs and their hierarchical structure (parent clubs and sub-clubs).

- **List view** — card grid showing Club ID, name, category, status badge (Active / Inactive), faculty coordinator, student lead, and member count.
- **Parent / Sub-club hierarchy** — a club can optionally have a `parentClubId` linking it to a parent club, supporting nested club structures.
- **Add / Edit form** — collects Club ID, Club Name, Category, Parent Club ID (optional), Description, Faculty Coordinator, Student Lead Name, Student Lead ID, Student Lead Role, Active Members, and Status.
- **View modal** — shows full club details in a read-only popup.
- **Delete** — confirmation dialog before deletion.

3. Sport Management

Manages university sports — each sport is linked to a venue and has a defined capacity and status.

- **List view** — card grid showing Sport ID, sport name, capacity, status badge (ACTIVE / INACTIVE), linked venue, and description.
- **Add / Edit form** — collects Sport ID, Sport Name, Capacity, Status, Venue ID, and Description.
- **Validation** — Sport ID required and unique; capacity must be ≥ 1; status must be ACTIVE or INACTIVE.
- **View modal** — read-only detail popup.
- **Delete** — confirmation before deletion.

4. Sport Teams

Manages teams within a sport — each team has a coach, captain, and a roster of students. Supports a join-request workflow.

- **List view** — table showing Team ID, Sport Name, Coach Name, Captain Name, and roster count.
- **Add / Edit form** — collects Team ID, Sport ID / Sport Name, Coach Name, and Captain Name.
- **Roster management** — inside the team detail view, students can be added to the roster by PRN lookup. Each roster entry stores PRN and student name. Students can be removed individually.
- **Join requests** — students can submit a join request (PENDING). The team manager can accept or ignore each request. Accepted requests auto-add the student to the roster.

- **URL-based navigation** — viewing a team detail changes the URL to `?team=T001` so refreshing the page restores the detail view correctly.
- **Delete** — confirmation before deletion.

5. Student Certificate

A multi-step workflow for generating official student certificates as PDF documents.

- **Select Student** — searchable table of all students seeded in the Spring Boot students collection. Paginated. Clicking a student navigates to the details page.
- **Student Details** — shows full student info (name, PRN, degree, year). User selects a certificate type from the available document types. Only one type can be selected at a time.
- **Certificate Preview** — renders the certificate as a PDF in an iframe using the selected document type's template, populated with the student's data. User can download the PDF.
- **Add Document Type** — CRUD page for managing certificate types (e.g. Bonafide, NOC). Each type has a name and a default content template. Existing types are listed with edit and delete options.
- **Handout** — bulk certificate generation. Select multiple students and multiple document types. Downloads a ZIP file containing one PDF per student × type combination.
- **Letterhead Editor** — single-document settings page (one record always exists with key = "default"). Edits college name, trust name, address, phone, toll-free, fax, website, email, and logo text. All certificates use this letterhead.

6. Study Materials

Two complementary pages — one for teachers to upload materials, one for students to browse and download them.

- **Upload Materials** — teacher-facing page. Step 1: select or create a folder. Step 2: drag-and-drop or click to select files (PDF, JPG, JPEG, PNG, XLS, XLSX). Each file shows upload progress. Files can be removed before uploading. Submit uploads all pending files to the selected folder.
- **Study Materials (Display)** — student/faculty-facing page. Left panel lists folders (subjects). Clicking a folder loads its files in the right panel as a table (filename, type badge, size, date). Files can be previewed inline (PDF/image) or downloaded.
- **Folders** — each folder belongs to a teacher. Folders can be created inline from the upload page. Sub-folder support via `parentFolderId`.
- **Role-based access** — upload and delete actions are restricted to TEACHER role via `X-User-Role` header. Students can only view and download.

7. Payment Management

Manages university fee payment titles and combinations. Allows admins to define individual fee items and bundle them into payment packages.

- **Payment Titles** — paginated table of individual fee items (Exam Fee, Library Fee, Sports Fee, etc.). Each title has an amount and a discount. Titles are seeded with 6 defaults on startup.
- **Add Payment Title** — inline form to add a new fee title with amount and discount.
- **Payment Combinations** — groups of selected titles bundled into a package. Total amount is auto-calculated as the sum of (amount − discount) for each included title.
- **Create Combination** — checkboxes on each title row let the user select items. Clicking "Add Payment" creates a combination from the selected titles. Duplicate combinations (same set of titles) are rejected.

8. Hostel Management

Manages hostel blocks, rooms within each block, and student-to-room allotments.

- **Block list** — card grid showing Block ID, hostel name, type badge (BOYS / GIRLS), and active/inactive status.
- **Add / Edit block** — modal form collecting Block ID, Hostel Name, Type, and Active status.
- **Block detail view** — clicking a block card navigates to a URL-based detail view (`?block=HB001`) that lists all rooms in that block. Refreshing the page restores the block detail correctly.
- **Room management** — within the block detail, rooms can be added (Room ID, Room Number, Capacity) and deleted. Each room shows the number of allotted students vs capacity.
- **Student allotment** — students are added to a room by PRN lookup (searches the Spring Boot students collection by PRN). Each room stores the student's PRN and name. Students can be removed from a room individually.
- **Delete block** — confirmation before deletion.

9. Venue Booking

Allows users to submit and manage venue booking requests for university events.

- **Booking request form** — auto-generates a Booking ID (format: VB-YYYYMMDD-XXXX). Dropdowns populated from the live Events API (Spring Boot) and Venues API (Node backend). Collects booking date (date picker, future dates only), start time, end time, purpose, and requested by.
- **Overlap validation** — the backend checks whether the selected venue is already booked (PENDING or APPROVED status) for the same date and overlapping time range. If a conflict exists, returns: *"This venue is already booked for the selected date and time."*

- **Booking list table** — shows all requests with columns: Booking ID, Event, Venue, Date, Start, End, Purpose, Requested By, Status, Actions.
- **Status values** — PENDING (default), APPROVED, REJECTED, CANCELLED — each displayed with a colour-coded badge.
- **Actions** — View (read-only modal with all details), Edit (modal to update fields and status), Delete (confirmation before deletion).
- **Status update** — a dedicated PATCH endpoint allows updating only the status without changing other fields.

Part 2 — MongoDB Schema Reference

Field-level schema definitions and example documents for the assigned feature modules. All collections use MongoDB (Spring Boot — Spring Data MongoDB).

1. Book Management

COLLECTION: BOOKS

Schema

Field	Type	Constraints
id	String	@Id — MongoDB auto-generated ObjectId
bookId	String	@Indexed(unique=true, sparse=true) — user-provided; existing books auto-assigned BK-001, BK-002...
bookTitle	String	@NotBlank("Book title is required")
authorName	String	@NotBlank("Author name is required")
totalCopies	Integer	@NotNull, @Min(0, "Total copies cannot be negative")
bookLocation	String	@NotBlank("Book location is required")
department	String	@NotBlank("Department is required")

Example Document

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e1",
  "bookId": "BK-001",
  "bookTitle": "Introduction to Algorithms",
  "authorName": "Thomas H. Cormen",
  "totalCopies": 10,
  "bookLocation": "Shelf A, Row 2",
  "department": "Computer Science"
}
```

bookId is entered by the user on the Add form and validated for uniqueness before saving. It is read-only on the Edit form. On startup, any existing book without a bookId is automatically assigned a sequential BK-NNN identifier.

2. Club Management

COLLECTION: CLUBS

Schema

Field	Type	Constraints
id	String	@Id — MongoDB auto-generated ObjectId
clubId	String	@Indexed(unique=true)
clubName	String	required
clubCategory	String	e.g. Technical, Cultural, Sports
parentClubId	String	default: null — null/empty = independent club; set to link as sub-club
description	String	optional
facultyCoordinator	String	optional
studentLeadName	String	optional
studentLeadId	String	optional
studentLeadRole	String	optional
activeMembers	Integer	optional
status	String	default: "Active" — values: Active, Inactive

Example Document

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e2",
  "clubId": "CLB001",
  "clubName": "Coding Club",
  "clubCategory": "Technical",
  "parentClubId": null,
  "description": "A club for competitive programming enthusiasts",
  "facultyCoordinator": "Dr. Priya Nair",
  "studentLeadName": "Arjun Mehta",
  "studentLeadId": "2021001",
  "studentLeadRole": "President",
  "activeMembers": 48,
  "status": "Active"
}
```

parentClubId links a sub-club to its parent. When null the club is a top-level independent club.

3. Sport Management

COLLECTION: SPORTS

Schema

Field	Type	Constraints
id	String	@Id — MongoDB auto-generated ObjectId
sportId	String	@Indexed(unique=true), @NotBlank("Sport ID is required")
sportName	String	@NotBlank("Sport name is required")
capacity	Integer	@NotNull, @Min(1, "Capacity must be greater than 0")
status	String	@NotBlank, @Pattern(regexp="ACTIVE|INACTIVE")
venueId	String	@NotBlank("Venue ID is required") — references Venue.venueId (Node collection)
description	String	optional

Example Document

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e3",
  "sportId": "SPT001",
  "sportName": "Cricket",
  "capacity": 22,
  "status": "ACTIVE",
  "venueId": "GRNDB-01",
  "description": "University cricket team, practices Tue/Thu"
}
```

4. Sport Teams

COLLECTION: SPORT_TEAMS

Schema — sport_teams

Field	Type	Constraints
id	String	@Id
teamId	String	@Indexed(unique=true)
sportId	String	references Sport.sportId
sportName	String	denormalized for display
coachName	String	required

Field	Type	Constraints
captainName	String	optional
roster	List<RosterEntry>	default: [] — embedded sub-documents
roster[].studentPrn	String	trimmed
roster[].studentName	String	trimmed

COLLECTION: TEAM_REQUESTS

Schema — team_requests

Field	Type	Constraints
id	String	@Id
requestId	String	@Indexed(unique=true)
teamId	String	references SportTeam.teamId
studentPrn	String	required
studentName	String	required
status	String	default: "PENDING" — values: PENDING, ACCEPTED, IGNORED

Example Document — sport_teams

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e4",
  "teamId": "T001",
  "sportId": "SPT001",
  "sportName": "Cricket",
  "coachName": "Mr. Rajesh Verma",
  "captainName": "Arjun Mehta",
  "roster": [
    { "studentPrn": "PRN2024001", "studentName": "Arjun Mehta" },
    { "studentPrn": "PRN2024002", "studentName": "Ravi Sharma" }
  ]
}
```

Example Document — team_requests

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e5",
  "requestId": "REQ001",
  "teamId": "T001",
  "studentPrn": "PRN2024003",
```

```
  "studentName": "Neha Desai",
  "status": "PENDING"
}
```

5. Student Certificate

COLLECTION: STUDENTS

Schema — students (Spring Boot certificate context)

Field	Type	Constraints
id	String	@Id
studentId	String	@Indexed(unique=true) — e.g. STU001
studentName	String	required
gender	String	e.g. Male, Female
prn	String	required — used for PRN lookup
division	String	e.g. A, B
yearOfEnrollment	String	e.g. 2021
studyingYear	String	e.g. Third Year
degreeProgramName	String	e.g. B.Tech Artificial Intelligence and Data Science
academicYear	String	e.g. 2026-27

COLLECTION: DOCUMENT_TYPES

Schema — document_types

Field	Type	Constraints
id	String	@Id
documentName	String	@Indexed(unique=true) — e.g. Bonafide Certificate, NOC
defaultContent	String	Template text with placeholders

COLLECTION: LETTERHEAD

Schema — letterhead (single document, key = "default")

Field	Type	Constraints
id	String	@Id

Field	Type	Constraints
key	String	always "default" — singleton document
trustName	String	required
collegeName	String	required
address	String	required
phone	String	optional
tollFree	String	optional
fax	String	optional
website	String	optional
email	String	optional
logoText	String	short text displayed in the logo area on generated PDFs

Example Document — letterhead

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e6",
  "key": "default",
  "trustName": "Your Trust / Management Name",
  "collegeName": "University ERP Institute of Technology",
  "address": "123 College Road, City - 000000, State, India",
  "phone": "+91-0000-000000",
  "tollFree": "1800-000-0000",
  "fax": "+91-0000-000001",
  "website": "www.university.edu",
  "email": "contact@university.edu",
  "logoText": "LOGO"
}
```

The letterhead collection always contains exactly one document with key = "default". All generated certificates and handouts use this document for their header.

6. Study Materials

COLLECTION: MATERIAL_FOLDERS

Schema — material_folders

Field	Type	Constraints
id	String	@Id

Field	Type	Constraints
folderId	String	unique identifier
folderName	String	required
teacherId	String	references the teacher who created the folder
parentFolderId	String	default: null — null = root-level folder
createdDate	LocalDateTime	auto-set on creation
updatedDate	LocalDateTime	auto-set on update

COLLECTION: MATERIALS

Schema — materials

Field	Type	Constraints
id	String	@Id
materialId	String	pattern: MAT-XXXXXXXX (8 hex chars)
fileName	String	display name; can be renamed by teacher
storedFileName	String	actual filename on disk; never changes after upload
fileType	String	enum: PDF, JPG, JPEG, PNG, XLS, XLSX
fileSize	long	file size in bytes
teacherId	String	references the uploader
folderId	String	references MaterialFolder.folderId
fileUrl	String	relative path: /api/materials/{id}/download
uploadedDate	LocalDateTime	auto-set on upload
updatedDate	LocalDateTime	auto-set on rename/update

Example Document — materials

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e7",
  "materialId": "MAT-A1B2C3D4",
  "fileName": "Data Structures Notes.pdf",
  "storedFileName": "ds-notes-20260901-143022.pdf",
  "fileType": "PDF",
  "fileSize": 1048576,
  "teacherId": "teacher@university.edu",
  "folderId": "FLD-001",
```

```
  "fileUrl": "/api/materials/64a1b2c3d4e5f6a7b8c9d0e7/download",
  "uploadedDate": "2026-09-01T14:30:22",
  "updatedDate": "2026-09-01T14:30:22"
}
```

PDF and image files are served with Content-Disposition: inline (preview in browser). XLS/XLSX and other types are served as attachment (forced download).

7. Payment Management

COLLECTION: PAYMENT_TITLES

Schema — payment_titles

Field	Type	Constraints
id	String	@Id
titleId	String	auto-generated — pattern: PT001, PT002...
title	String	required — e.g. "Exam Fee", "Library Fee"
amount	double	required — gross fee amount in INR
discount	double	required — discount value in INR (not a percentage)

COLLECTION: PAYMENT_COMBINATIONS

Schema — payment_combinations

Field	Type	Constraints
id	String	@Id
paymentId	String	auto-generated — pattern: PAY001, PAY002...
paymentTitles	List<String>	sorted list of title names included in this combination
totalAmount	double	computed as Σ(amount − discount) for each included title
createdAt	LocalDateTime	auto-set on creation

Example Document — payment_titles

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e8",
  "titleId": "PT001",
  "title": "Exam Fee",
  "amount": 2000.0,
```

```
  "discount": 100.0
}
```

Example Document — payment_combinations

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0e9",
  "paymentId": "PAY001",
  "paymentTitles": ["Exam Fee", "Library Fee", "Sports Fee"],
  "totalAmount": 3250.0,
  "createdAt": "2026-09-01T10:00:00"
}
```

6 default payment titles are seeded automatically on every Spring Boot startup. totalAmount is computed server-side and never entered by the user.

8. Hostel Management

COLLECTION: HOSTEL_BLOCKS

Schema — hostel_blocks

Field	Type	Constraints
id	String	@Id
blockId	String	@Indexed(unique=true) — e.g. HB001, HB002
hostelName	String	required — e.g. "Shivaji Block"
type	String	required — values: BOYS, GIRLS
active	boolean	default: true

COLLECTION: HOSTEL_ROOMS

Schema — hostel_rooms

Field	Type	Constraints
id	String	@Id
roomId	String	unique — e.g. HR001, HR002
blockId	String	references HostelBlock.blockId
roomNo	String	e.g. "101", "A-12"
capacity	int	required — max students per room

Field	Type	Constraints
students	List<RoomStudent>	default: [] — embedded sub-documents
students[].studentPrn	String	trimmed
students[].studentName	String	trimmed

Example Document — hostel_blocks

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0ea",
  "blockId": "HB001",
  "hostelName": "Shivaji Block",
  "type": "BOYS",
  "active": true
}
```

Example Document — hostel_rooms

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0eb",
  "roomId": "HR001",
  "blockId": "HB001",
  "roomNo": "101",
  "capacity": 3,
  "students": [
    { "studentPrn": "PRN2024001", "studentName": "Arjun Mehta" },
    { "studentPrn": "PRN2024002", "studentName": "Ravi Sharma" }
  ]
}
```

Students are added to a room by PRN lookup against the Spring Boot students collection. The students array stores a denormalized copy of PRN and name for fast rendering.

9. Venue Booking

COLLECTION: VENUE_BOOKINGS

Schema

Field	Type	Constraints
id	String	@Id — MongoDB auto-generated ObjectId
bookingId	String	@Indexed(unique=true), @NotBlank — auto-generated as VB-YYYYMMDD-XXXX

Field	Type	Constraints
eventId	String	@NotBlank("Event ID is required") — references Spring Boot events collection
venueId	String	@NotBlank("Venue ID is required") — references Node backend venues collection
bookingDate	String	@NotNull("Booking date is required") — stored as "YYYY-MM-DD"
startTime	String	@NotBlank("Start time is required") — stored as "HH:mm"
endTime	String	@NotBlank("End time is required") — stored as "HH:mm"; must be > startTime
purpose	String	optional — free text description
requestedBy	String	@NotBlank("Requested by is required")
status	String	@Pattern(regexp="PENDING|APPROVED|REJECTED|CANCELLED"); default: "PENDING"

Example Document

```
{
  "_id": "64a1b2c3d4e5f6a7b8c9d0ec",
  "bookingId": "VB-20260902-7483",
  "eventId": "EVT260001",
  "venueId": "HALL-001",
  "bookingDate": "2026-11-15",
  "startTime": "09:00",
  "endTime": "13:00",
  "purpose": "Annual Sports Day Opening Ceremony",
  "requestedBy": "Prof. Sharma",
  "status": "PENDING"
}
```

The overlap check queries for any existing booking on the same venueId and bookingDate where the time ranges intersect AND status is PENDING or APPROVED. CANCELLED and REJECTED bookings are excluded from the conflict check.